QuDPy-FDGF Solver API and Implementation Reference

Mathieu Desmarais, Simón Paiva-Ortega, Hao Li, Eric R. Bittner, Carlos Silva-Acuña

Software version 1.0.0

Abstract

This document collects the software-facing reference material for the QuDPy-FDGF solver. It describes the model contracts, public solver operations, observable specifications, pathway objects, numerical backends, and plotting utilities. The scientific manuscript contains the mathematical formulation, validation benchmarks, and physical examples; this companion document records API details that may evolve with the software.

Contents

1 Quick start **3**
- 1.1 Installation . . . 3
- 1.2 A complete calculation . . . 3
- 1.3 Expected output . . . 5
- 1.4 Next steps and tests . . . 5

2 Solver API and implementation reference **5**
- 2.1 `SectorModel`: external model contract . . . 5
  - 2.1.1 Required structural methods . . . 6
  - 2.1.2 Initial and equilibrium states . . . 6
  - 2.1.3 Transition, detection, and collapse operators . . . 7
  - 2.1.4 Requirements and capabilities . . . 7
- 2.2 `EigenbasisKModel`: provided k-space adapter . . . 7
  - 2.2.1 Constructor inputs . . . 8
  - 2.2.2 Diagonalization and basis transformation . . . 8
  - 2.2.3 Automatic transition-operator decomposition . . . 8
  - 2.2.4 Detection and named observables . . . 9
  - 2.2.5 Initial and equilibrium states . . . 9
  - 2.2.6 Dissipative channels and backend requirements . . . 10
- 2.3 `ExcitationSectorModel`: explicit manifold adapter . . . 10
  - 2.3.1 Constructor and block convention . . . 10
  - 2.3.2 Initial and thermal states . . . 11
- 2.4 `SpectroscopySolver`: numerical orchestrator . . . 11
  - 2.4.1 Constructor . . . 11
  - 2.4.2 Model registration and backend construction . . . 12
  - 2.4.3 Reference-state check . . . 12
  - 2.4.4 Stationary state of the generator . . . 13
  - 2.4.5 Pathway registration . . . 14
  - 2.4.6 Propagation protocols . . . 14

2.4.7 Single-pathway calculation . . . 15
2.4.8 Several observables from one pathway propagation . . . 15
2.4.9 Component calculation . . . 16
2.4.10 Multidimensional spectrum generation . . . 16
2.4.11 Evaluation strategies . . . 17
2.4.12 Coherence-order-resolved-spectrum . . . 18
2.4.13 Result containers . . . 19
2.4.14 Jump-channel access . . . 19
2.4.15 Decay-rate diagnostic . . . 19
2.4.16 Direct Hilbert-space resolvent . . . 19
2.4.17 Memory estimation . . . 20
2.4.18 Numerical backend architecture . . . 20
2.4.19 Solver Summary . . . 26
2.5 `ObservableSpec`: operator, jump, and action detection . . . 27
2.5.1 Named operator observables . . . 27
2.5.2 Instantaneous and integrated jump observables . . . 27
2.5.3 Action detection through a fourth interaction . . . 28
2.5.4 Normalization and aliases . . . 29
2.6 **Pathway conventions** . . . 29
2.6.1 Interaction fields . . . 29
2.6.2 Coherence-order history . . . 30
2.6.3 FrequencyPathway fields . . . 30
2.6.4 Pathway response prefactor . . . 31
2.6.5 Signatures and completeness . . . 31
2.6.6 Vanishing pathways . . . 32
2.6.7 UFSS diagram translation . . . 32
2.7 `SpectroscopyPlotter` . . . 33
2.7.1 Constructor . . . 33
2.7.2 Selecting data from a `SpectrumResult` . . . 33
2.7.3 Dictionary-based plotting configuration . . . 34
2.7.4 Views, normalization, and guides . . . 34

Notation and conventions

The notation follows the companion manuscript. Operators on the Hilbert space carry hats ($\hat{H}$, $\hat{\rho}$, $\hat{\mu}$, $\hat{L}_c$, $\hat{O}$); calligraphic symbols denote superoperators ($\mathcal{L}, \mathcal{U}, \mathcal{R}_\eta, \mathcal{V}, \mathcal{D}_c$), and the same symbol denotes a superoperator and its matrix acting on the column-stacked vector $\mathrm{vec}(\hat{\rho}) = |\hat{\rho}\rangle\!\rangle$; $\mathbb{1}$ is the identity. Dissipative channels are labeled by c, with jump operator $\hat{L}_c$, dissipator $\mathcal{D}_c[\hat{\rho}] = \hat{L}_c\hat{\rho}\hat{L}_c^\dagger - \frac{1}{2}\{\hat{L}_c^\dagger\hat{L}_c, \hat{\rho}\}$, and rate γ_c kept separate: $\mathcal{L}[\hat{\rho}] = -i[\hat{H}, \hat{\rho}] + \sum_c \gamma_c \mathcal{D}_c[\hat{\rho}]$. The rate γ_c is the Lindblad coefficient, not necessarily a measured rate: a pure-dephasing rate $\gamma_\phi = 1/T_2^*$ requires $\gamma_c = \gamma_\phi/2$ for $\hat{L}_c = \hat{\sigma}_z$ and $\gamma_c = 2\gamma_\phi$ for a projector $|e\rangle\langle e|$ (Sec. 1). The raising and lowering parts of the light–matter operator are $\hat{\mu}_\pm$; in the software interface they are supplied as the transition blocks $J_\pm$ (`j_plus_array`, `j_minus_array`). The pathway prefactor is c_p and the detection efficiency $\varepsilon_{\mathrm{det}}$; η is reserved for the resolvent regularization. Energies and frequencies share one unit, $\hbar = 1$ (eV in the examples).

1 Quick start

1.1 Installation

QuDPy-FDGF requires Python $\geq$ 3.10. The distribution is named `qudpy-fdgf` and the import name is `qudpy_fdgf`. From a clone of the repository, install it in editable mode:

```
git clone https://github.com/SilvaScience/QuDPy-FDGF.git
cd QuDPy-FDGF
pip install -e .              # core: NumPy, SciPy, Matplotlib
pip install -e ".[examples]"  # optional: QuTiP, UFSS, IPython (notebooks)
pip install -e ".[dev]"       # optional: pytest
```

The core dependencies are NumPy $\geq$ 1.24, SciPy $\geq$ 1.10, and Matplotlib $\geq$ 3.7; Matplotlib is used only by `SpectroscopyPlotter`. QuTiP is used by the example notebooks to build operators, and `ufss.DiagramGenerator` generates the fifth-order pathways of the chromophore example. The optional extra `[manybody]` installs TeNPy for the experimental modules, which are not part of this release. The reference output below was obtained with Python 3.13.14, NumPy 2.5.0, SciPy 1.18.0, and Matplotlib 3.11.0.

1.2 A complete calculation

The script below treats an open two-level system with transition energy 2 eV, radiative decay at rate γ_1, and pure dephasing at rate γ_ϕ. It performs the five steps common to every calculation: build the matrices, adapt them to the model contract, feed the model to the solver, declare the pathways and the protocol, and generate the spectrum.

```
import numpy as np
from qudpy_fdgf import EigenbasisKModel, FrequencyPathway, SpectroscopySolver
from qudpy_fdgf.protocols import (PropagationInterval, SpectroscopyProtocol,
                                  standard_nq_protocol)

# 1. Matrices (hbar = 1, eV), basis ordering (|e>, |g>)
sigma_z = np.diag([1.0, -1.0]).astype(complex)
sigma_x = np.array([[0, 1], [1, 0]], dtype=complex)
sigma_m = np.array([[0, 0], [1, 0]], dtype=complex)   # |g><e|
```

```
H, mu = 1.0 * sigma_z, 0.5 * sigma_x
gamma_1, gamma_phi = 0.010, 0.005                    # 1/T1, 1/T2* (eV)

# 2. Model contract: (operator, rate) pairs, rate kept out of the operator
model = EigenbasisKModel(H, mu, c_ops_raw=(
    (sigma_m, gamma_1),          # population decay at gamma_1
    (sigma_z, gamma_phi / 2),    # sigma_z dephases coherences at 2x its rate
))

# 3. Solver
solver = SpectroscopySolver(backend="dense", eta=1e-4)
solver.feed_model(model)

# 4-5. Linear response: one ket interaction, one frequency interval
linear = FrequencyPathway("linear", interactions=[{"label": "Ku"}],
                          component="linear")
protocol = SpectroscopyProtocol(
    intervals=[PropagationInterval("omega1", "frequency")])
w = np.linspace(1.9, 2.1, 4001)
res = solver.generate_spectrum(protocol, {"omega1": w}, pathways=[linear])
A = res.pathways["linear"].imag      # absorptive part
half = w[A >= A.max() / 2]
Gamma_2 = gamma_1 / 2 + gamma_phi
print(f"linear peak      : {w[np.argmax(A)]:.4f} eV")
print(f"FWHM (numerical) : {half[-1] - half[0]:.4f} eV")
print(f"2(Gamma_2 + eta) : {2 * (Gamma_2 + solver.eta):.4f} eV")

# Third-order rephasing (R1 + R2) at t2 = 0: omega_1 and omega_3 resolved
R1 = FrequencyPathway("R1", interactions=[
    {"label": "Bu", "pulse_index": 0}, {"label": "Ku", "pulse_index": 1},
    {"label": "Bd", "pulse_index": 2}], component="rephasing")
R2 = FrequencyPathway("R2", interactions=[
    {"label": "Bu", "pulse_index": 0}, {"label": "Bd", "pulse_index": 1},
    {"label": "Ku", "pulse_index": 2}], component="rephasing")
protocol_2d = standard_nq_protocol(order=1, nq_interval=1, detection_interval=3,
                                   n_interactions=3, nq_axis="omega1q")
axes = {"omega1q": np.linspace(-2.2, -1.8, 81),
        "omega3": np.linspace(1.8, 2.2, 81)}
res2 = solver.generate_spectrum(protocol_2d, axes, pathways=[R1, R2],
                                fixed_coordinates={"t2": 0.0})
S2 = res2.components["rephasing"]
i, j = np.unravel_index(np.argmax(np.abs(S2)), S2.shape)
print(f"rephasing peak   : (omega1q, omega3) = "
      f"({res2.axis_values[0][i]:.3f}, {res2.axis_values[1][j]:.3f}) eV")
print(f"|S| at peak      : {np.abs(S2[i, j]):.2f}")
print(f"2 mu^4/(Gamma_2 + eta)^2 : {2 * 0.5**4 / (Gamma_2 + solver.eta)**2:.2f}")
```

1.3 Expected output

The script runs in about two seconds on a laptop and prints

```
linear peak      : 2.0000 eV
FWHM (numerical) : 0.0202 eV
2(Gamma_2 + eta) : 0.0202 eV
rephasing peak   : (omega1q, omega3) = (-2.000, 2.000) eV
|S| at peak      : 1225.37
2 mu^4/(Gamma_2 + eta)^2 : 1225.37
```

Each numerical line is followed by its analytical value. The absorptive line has a full width at half maximum $2(\Gamma_2 + \eta)$, with $\Gamma_2 = \gamma_1/2 + \gamma_\phi$; the width confirms that each supplied rate γ_c multiplies $\mathcal{D}_c[\hat{\rho}]$ unchanged, so that the $\hat{\sigma}_z$ channel must carry $\gamma_\phi/2$ (a diagonal channel $\hat{L}_c = \sum_x l_x \, |x\rangle \langle x|$ damps $|x\rangle \langle y|$ at the rate $\gamma_c(l_x - l_y)^2/2$). The rephasing signal peaks on the antidiagonal quadrant $(\omega_{1q}, \omega_3) = (-2, 2)$ eV, where each of the two pathways contributes $|\hat{\mu}_{eg}|^4/(\Gamma_2 + \eta)^2$. Replacing `backend="dense"` by `backend="sparse_sector"` gives the same output.

1.4 Next steps and tests

- `examples/quickstart.py`: the listing above.
- `examples/example1_two_level_open.ipynb`: Example 1 of the manuscript, with populations, action detection, and integrated jump observables (Sec. 2.5); the listing of the manuscript is extracted from its cells tagged `listing`.
- `examples/example2_xxz_chain_thermal.ipynb` and `examples/example3_chi5_2q.ipynb`: Examples 2 and 3.
- `python -m pytest`: runs the test suite in `tests/`, including the analytical checks of the manuscript (about 17 s with a single BLAS thread, `OMP_NUM_THREADS=1`); `tests/api/` checks the contracts and options of the library, `tests/physics/` the closed-form results and limiting cases, and `tests/regression/` compares eight small calculations with stored outputs, which any change of the numerical core must reproduce.
- `validation/`: analysis notebooks (convergence, analytical limits, backend agreement), benchmark notebooks and scripts (time, memory, comparison with the original QuDPy), the scripts that redraw the figures of the manuscript, the shared models, and the stored results (see `validation/README.md`).

Section 2 documents the model contract, the solver, and the backends; Sec. 2.6 fixes the sign and ordering conventions of the pathways.

2 Solver API and implementation reference

2.1 `SectorModel`: external model contract

`SectorModel` is the block-based interface between an external physical model and `SpectroscopySolver`. The model defines the physical truncation, basis, operators, initial

state, and dissipative channels; the solver validates, assembles, and propagates these objects. The truncated Hilbert space is a direct sum,

$$\mathcal{H} = \bigoplus_{s \in \mathcal{S}} \mathcal{H}_s, \qquad \dim(\mathcal{H}_s) = d_s, \tag{1}$$

and a block mapping source sector s to target sector t has the form

$$\hat{A}_{ts} = \hat{P}_t \hat{A} \hat{P}_s \in \mathbb{C}^{d_t \times d_s}. \tag{2}$$

Sector labels are opaque hashable objects and sector dimensions need not be equal.

2.1.1 Required structural methods

The minimal contract used by `feed_model` is

- `sectors()` and `dimension(sector)`;
- `hamiltonian_blocks(source)`, returning a mapping from target sectors to Hamiltonian blocks;
- `transition_blocks(...)`, with source sector, operator name, and direction (`"plus"` or `"minus"`);
- `observable_blocks(...)`, returning the blocks of a named detection operator for a source sector.

Hamiltonian blocks may be diagonal or inter-sector. Operator blocks may be dense arrays, sparse matrices, or matrix-free linear operators exposing their shape and forward/adjoint actions. The backends apply a block to the whole density matrix at once, as a product with the matrix of all its columns. A linear operator should therefore define `matmat` and `rmatmat` when it can do better than a loop over columns (every `scipy.sparse.linalg.LinearOperator` has them, with that loop as default); an object that provides only `matvec` and `rmatvec` is still accepted and applied column by column.

2.1.2 Initial and equilibrium states

The model method `initial_condition(context=None)` returns either a `PureState` or a `DensityState`. A pure state stores one sector label and a vector in that sector. A density state stores blocks under keys `(ket_sector, bra_sector)`:

$$\rho_{ab} = \hat{P}_a \hat{\rho} \hat{P}_b \in \mathbb{C}^{d_a \times d_b}. \tag{3}$$

Both populations ($a = b$) and cross-sector coherences ($a \neq b$) are supported. A block may be supplied as a dense matrix through `DenseDensityBlock`, or in factored form through `LowRankDensityBlock(X,Y,S)`, representing

$$\rho_{ab} = X S Y^{\dagger}, \tag{4}$$

with S optional. During backend construction, the assembled state is normalized and checked for compatible dimensions, Hermiticity, unit trace, and positive semidefiniteness within numerical tolerance.

Thermodynamic input is normalized to

```
ThermodynamicContext(
    temperature,
    ensemble="canonical",
    chemical_potentials={},
    k_ensemble="independent",
    metadata={},
)
```

The supported ensemble labels are canonical and grand canonical; the k ensemble is independent or global. A missing context does not imply zero temperature or equilibrium: it delegates the default state to the model. When a context is supplied, the optional equilibrium-state method remains model-side; the solver does not infer a thermal state from the Hamiltonian. The provided adapters build the Gibbs state of $\hat{H}$ without reference to the collapse channels; whether it is stationary under the generator is checked by the solver (Sec. 2.4.3).

2.1.3 Transition, detection, and collapse operators

For a named interaction operator, the model supplies the physical decomposition

$$\hat{\mu} = \hat{\mu}_+ + \hat{\mu}_-. \tag{5}$$

The solver does not infer this decomposition from energy differences or sector labels. The separate `observable_blocks` method exposes named detection operators, so propagation and readout operators need not coincide.

The backends check the operators they receive. When they are built, $\hat{H}x$ and $\hat{H}^\dagger x$ are compared for two random vectors, and a non-Hermitian Hamiltonian raises `ModelContractError` (backend option `hermiticity_tolerance`, default 10^{-10}, relative; the check is skipped for matrix-free blocks that provide no adjoint action). At the first use of each interaction operator, $\hat{\mu}_- x$ is compared with $\hat{\mu}_+^\dagger x$, and a mismatch emits a `ModelConsistencyWarning` (option `transition_tolerance`, default 10^{-8}), because $\hat{\mu} = \hat{\mu}_+ + \hat{\mu}_-$ is then not Hermitian and the detected signal is not that of a physical observable. The adapters supplied with the package satisfy both conditions by construction.

The optional collapse-channel method returns `CollapseChannel` objects. A channel stores a name, a non-negative rate γ_c, and blockwise $\hat{L}_c$, with the convention

$$\mathcal{L}[\hat{\rho}] = -i[\hat{H}, \hat{\rho}] + \sum_c \gamma_c \mathcal{D}_c[\hat{\rho}], \qquad \mathcal{D}_c[\hat{\rho}] = \hat{L}_c \hat{\rho} \hat{L}_c^\dagger - \frac{1}{2}\{\hat{L}_c^\dagger \hat{L}_c, \hat{\rho}\}. \tag{6}$$

The rate must therefore not be folded into the supplied operator blocks.

2.1.4 Requirements and capabilities

The optional `requirements()` and `capabilities()` methods advertise the requested state kind, generator kind, propagation domains, exactness, matrix-free constraint, and related metadata. The solver combines this declaration with the effective initial state and collapse channels before selecting or validating a backend. Thus `SectorModel` is a data contract, not a prescribed physical model.

2.2 `EigenbasisKModel`: provided k-space adapter

`EigenbasisKModel` constructs a `SectorModel`-compatible object from site-basis Hamiltonians and operators. A single (d, d) Hamiltonian produces one sector; an (N_k, d, d) stack produces independent sectors $\{k_0, \ldots, k_{N_k-1}\}$. This adapter introduces no inter-k coupling.

2.2.1 Constructor inputs

The constructor has the form

```
EigenbasisKModel(
    H_model,
    interaction_op_array,
    *,
    c_ops_raw=(),
    detection_op_array=None,
    observable_op_arrays=None,
    j_plus_array=None,
    j_minus_array=None,
    rwa_tol=1e-6,
    transition_window=None,
    k_weights=None,
    energy_unit=None,
    boltzmann_constant=None,
)
```

2.2.2 Diagonalization and basis transformation

For each k-point, the adapter diagonalizes the site-basis Hamiltonian,

$$\hat{H}_{site}^{(k)} U^{(k)} = U^{(k)} E^{(k)}. \tag{7}$$

where $E^{(k)}$ is diagonal. Every supplied interaction, detection, observable, and collapse operator is transformed according to

$$\hat{A}_{\text{eig}}^{(k)} = \left(U^{(k)}\right)^{\dagger} \hat{A}_{\text{site}}^{(k)} U^{(k)}. \tag{8}$$

These transformations are performed once during model construction.

2.2.3 Automatic transition-operator decomposition

When explicit `j_plus_array` and `j_minus_array` inputs are not provided, the transformed interaction operator is separated according to the eigenenergy differences.

For a matrix element connecting source eigenstate j to target eigenstate i, define

$$\Delta E_{ij}^{(k)} = E_i^{(k)} - E_j^{(k)} \tag{9}$$

The automatic decomposition is

$$\left(\hat{\mu}_{+}^{(k)}\right)_{ij} = \begin{cases} \left(\hat{\mu}^{(k)}\right)_{ij}, & \Delta E_{ij}^{(k)} > \texttt{rwa_tol}, \\ 0, & \text{otherwise}, \end{cases} \tag{10}$$

$$\left(\hat{\mu}_{-}^{(k)}\right)_{ij} = \begin{cases} \left(\hat{\mu}^{(k)}\right)_{ij}, & \Delta E_{ij}^{(k)} < -\texttt{rwa_tol}, \\ 0, & \text{otherwise}. \end{cases} \tag{11}$$

Matrix elements satisfying

$$|\Delta E_{ij}^{(k)}| \leq \text{rwa_tol} \tag{12}$$

are omitted from both components; this includes the static dipole moments and the elements between degenerate states. Models requiring explicit manifold assignments, degeneracy handling, or external pathway conventions may instead provide both `j_plus_array` and `j_minus_array`. The reported decomposition is then `"automatic_energy"` or `"explicit"`.

Bandwidth window and report. In the impulsive limit every transition of positive energy is treated as resonant with the field, however small its Bohr frequency, which is wrong for a low-frequency transition (vibrational, or intra-manifold) of a model driven by an optical pulse. The argument `transition_window=`$(\omega_{\min}, \omega_{\max})$ keeps only the elements with $\omega_{\min} \leq |\Delta E_{ij}| \leq \omega_{\max}$, in the units of the Hamiltonian: the band that the pulses of the experiment can drive. It requires $0 \leq \omega_{\min} < \omega_{\max}$ and cannot be combined with explicit `j_plus_array` and `j_minus_array`. The method `transition_summary()` returns a dictionary with the fractions of $\sum_{ij} |\mu_{ij}|^2$ that are kept (`kept_fraction`), static (`static_fraction`: diagonal elements of non-degenerate states), inside degenerate subspaces (`degenerate_fraction`) and excluded by the window (`outside_window_fraction`), together with `n_degenerate_states`; the degenerate weight is the Frobenius weight of the zero-frequency blocks, which does not depend on the basis chosen inside a degenerate subspace. If the degenerate weight exceeds 10^{-6} of the total, the constructor emits a `ModelConsistencyWarning`: the field does not couple these states, and the explicit arrays must be used if it should. `ExcitationSectorModel.transition_summary()` reports `"explicit_sector"`.

2.2.4 Detection and named observables

For every sector k, the adapter returns only intra-sector blocks:

$$\texttt{hamiltonian_blocks(k)} = \{\texttt{k} : \hat{H}^{(k)}_{\text{eig}}\}, \tag{13}$$

$$\texttt{transition_blocks("light_matter","plus",k)} = \{\texttt{k} : \hat{\mu}^{(k)}_{+}\}, \tag{14}$$

$$\texttt{transition_blocks("light_matter","minus",k)} = \{\texttt{k} : \hat{\mu}^{(k)}_{-}\}. \tag{15}$$

The default detection operator is the transformed `detection_op_array`; if it is omitted, the interaction operator is reused. The observable mapping adds independently named operators, transformed in the same way. Their names are returned by `observable_names()`. When named observables are configured, requesting an unknown name raises an error; without that mapping, arbitrary names retain the legacy fallback to the single detection operator.

2.2.5 Initial and equilibrium states

The default initial condition is a mixed state distributed over the k sectors according to normalized `k_weights`. Within each sector, the population occupies the lowest eigenstate:

$$\rho^{(0)}_{kk} = w_k \left|0_k\right\rangle \left\langle 0_k\right|, \qquad \sum_k w_k = 1. \tag{16}$$

For a supplied thermodynamic context, the equilibrium state is constructed from the eigenergies of each sector. At positive temperature,

$$\rho^{\text{eq}}_{kk} = w_k \sum_i \frac{\exp\left[-\beta E^{(k)}_i\right]}{Z_k} \left|i_k\right\rangle \left\langle i_k\right|, \qquad Z_k = \sum_i \exp\left[-\beta E^{(k)}_i\right], \tag{17}$$

with

$$\beta = \frac{1}{k_B T} \tag{18}$$

where k_B must be in the unit of the Hamiltonian per kelvin. It has no default, because a wrong unit gives a wrong thermal state without any warning: pass `energy_unit` (Sec. 2.3.2) or `boltzmann_constant` to the model, otherwise a temperature above zero raises `ValueError` at `feed_model`. At zero temperature, degenerate lowest states within each sector share its weight. The present adapter treats the k sectors independently and preserves the fixed k weights.

2.2.6 Dissipative channels and backend requirements

Each raw collapse-operator pair $(\hat{L}_c, \gamma_c)$ is transformed into the eigenbasis of every k-sector. The resulting channel has a generated name such as `c_op_0` and contains only intra-sector blocks,

$$\hat{L}^{(k)}_{c,\mathrm{eig}} = \left(U^{(k)}\right)^{\dagger} \hat{L}^{(k)}_{c,\mathrm{site}} U^{(k)}. \tag{19}$$

If at least one collapse channel is supplied, the adapter advertises a Lindblad generator; otherwise it advertises unitary evolution. Its default state representation is mixed because the independent k sectors are combined incoherently.

2.3 `ExcitationSectorModel`: explicit manifold adapter

`ExcitationSectorModel` is the adapter for models already organized into excitation or other user-defined sectors. Unlike `EigenbasisKModel`, sector dimensions may differ and rectangular inter-sector operator blocks are supported explicitly.

2.3.1 Constructor and block convention

```
ExcitationSectorModel(
    hamiltonian_blocks,
    raising_blocks,
    *,
    lowering_blocks=None,
    detection_blocks=None,
    observable_op_blocks=None,
    c_ops_raw=(),
    initial_sector=None,
    initial_state_index=0,
    energy_unit=None,
    boltzmann_constant=None,
)
```

The Hamiltonian mapping contains one dense Hermitian block per sector. All operator mappings use `(target, source)` sector keys. Each sector Hamiltonian is diagonalized as

$$\hat{H}_s V_s = V_s E_s, \tag{20}$$

and every rectangular block is transformed consistently,

$$\hat{A}^{\mathrm{eig}}_{ts} = V_t^{\dagger} \hat{A}_{ts} V_s. \tag{21}$$

If `lowering_blocks` is omitted, it is generated as the blockwise adjoint of `raising_blocks`. If `detection_blocks` is omitted, the sum of raising and lowering blocks is used. The mapping `observable_op_blocks` exposes additional named observables through the same block convention. Because the transition blocks are supplied directly, `transition_decomposition()` reports `"explicit_sector"`.

2.3.2 Initial and thermal states

The default initial sector is the one containing the global lowest eigenenergy unless `initial_sector` is specified; `initial_state_index` selects the eigenstate inside that sector. For a thermodynamic context, the adapter constructs one global canonical distribution over all retained sector eigenstates,

$$\hat{\rho}^{\mathrm{eq}} = \frac{1}{Z} \sum_{s,i} \exp(-\beta E_{si}) \, |s,i\rangle \langle s,i| \,, \qquad Z = \sum_{s,i} \exp(-\beta E_{si}). \tag{22}$$

At zero temperature, the population is shared among all globally degenerate ground states. The Boltzmann constant again fixes the energy–temperature unit conversion and again has no default (Sec. 2.3.2).

Energy unit and Boltzmann constant. Both adapters take `energy_unit=None` and `boltzmann_constant=None`. A thermal state ($T > 0$) needs k_B in the unit of the Hamiltonian per kelvin, given by one of the two arguments:

`energy_unit`	unit of the Hamiltonian	k_B per kelvin
`"eV"`	electronvolt	$8.617333262 \times 10^{-5}$
`"meV"`	millielectronvolt	$8.617333262 \times 10^{-2}$
`"cm-1"`	wavenumber	0.6950348004
`"THz"`	ordinary frequency E/h	$2.083661912 \times 10^{-2}$

(CODATA 2018.) `"THz"` is the ordinary frequency $\nu = E/h$, so $k_B/h = 0.0208\,\mathrm{THz/K}$; a Hamiltonian in angular frequency (rad/ps) needs $k_B/\hbar = 0.1309\,\mathrm{rad\,ps^{-1}/K}$ and is passed through `boltzmann_constant`. Giving both arguments, an unknown unit or a non-positive, infinite or NaN constant raises `ValueError`. With neither, a temperature of zero is accepted, since no k_B enters, and a temperature above zero raises a `ValueError` that lists the options when the model is registered with `feed_model`. A model written by the user supplies its own thermal state and is not affected.

2.4 `SpectroscopySolver`: numerical orchestrator

`SpectroscopySolver` is the public orchestrator of the solver. It connects an external SectorModel to a numerical backend, manages the active pathways responses, and assembles multidimensional spectra. The solver does not contain model-specific physics. The physical information is supplied by the SectorModel, while the numerical propagation is delegated to the selected backend.

2.4.1 Constructor

```
SpectroscopySolver(
    backend="sparse_sector",
    eta=...,                    # required, no default
    requirements=None,
```

```
    thermodynamic_context=None,
    check_stationarity="warn",
    stationarity_tolerance=1e-8,
    **backend_options
)
```

The broadening `eta` is required and has no default. It is the homogeneous linewidth of the frequency-domain resolvents $[(\eta - i\omega)\mathbb{1} - \mathcal{L}]^{-1}$ (a line of decay rate Γ becomes a Lorentzian of half width $\Gamma + \eta$) and is expressed in the unit of the Hamiltonian, so that no number is reasonable for every model. A missing `eta`, or a negative, infinite or NaN one, raises `ValueError`, from the solver and from the backends alike. The arguments `check_stationarity` and `stationarity_tolerance` control the check of the reference state described below.

When `backend="auto"` is selected, the solver compares the effective model requirements with the declared capabilities of the registered backends. Built-in and custom backends are listed by `available_backends()` and custom implementations are added with `register_backend(name, backend_class)`.

2.4.2 Model registration and backend construction

The method `feed_model(model, context=None, initial_state=None)` connects a `SectorModel` to the solver. The method performs the following operations:

1. validates the structural model contract;
2. resolves the thermodynamic context, initial state, and collapse channels;
3. updates the effective state and generator requirements;
4. selects or validates a compatible numerical backend;
5. constructs the sector layout, generator, and operator blocks;
6. checks that the reference state is stationary (Sec. 2.4.3).

The optional `context` argument overrides the thermodynamic context provided to the constructor when it is not `None`. The method return the solver instance itself, allowing chained construction:

```
solver = (
SpectroscopySolver(backend="sparse_sector", eta=0.002)
.feed_model(model, context=context))
```

Pathway calculations cannot be performed before `feed_model` has successfully constructed the backend.

2.4.3 Reference-state check

The perturbative response of the manuscript starts from a reference state $\hat{\rho}_{\mathrm{ref}}$ that does not evolve before the first pulse, $\mathcal{L}\hat{\rho}_{\mathrm{ref}} = 0$. The model supplies this state and nothing in the Hamiltonian guarantees the property: the Gibbs state of $\hat{H}$ is stationary for a closed model, but with collapse channels it is stationary only if the channels satisfy the detailed balance of $\hat{H}$ at that temperature.

At the end of `feed_model` the solver therefore applies the generator once to the reference state and computes the relative residual

$$r = \frac{\|\mathcal{L}|\hat{\rho}_{\mathrm{ref}}\rangle\!\rangle\|}{\||\hat{\rho}_{\mathrm{ref}}\rangle\!\rangle\|}, \tag{23}$$

with Frobenius norms and in the units of the Hamiltonian and of the rates. The constructor argument `check_stationarity` selects the behavior when r exceeds `stationarity_tolerance` (default 10^{-8}): `"warn"` (default) emits a `StationarityWarning`, `"error"` raises `StationarityError`, and `"off"` skips the check. The method `stationarity_residual()` returns r at any time, and `summary()` reports it. For example, an open two-level system at 10 K with the channel $\hat{L} = |g\rangle\langle e|$ has a Gibbs state with 9% of excited population that the channel empties: $r = 0.03$, and the linear response at resonance is 9% below that of the stationary state, which is the ground state. The check does not construct a stationary state; a state that fails it is replaced by the stationary state of the generator (Sec. 2.4.4), computed by `steady_state()` and imposed with `feed_model(model, initial_state="steady")`.

2.4.4 Stationary state of the generator

The reference state can be replaced by the stationary state of the generator, $\mathcal{L}\hat{\rho}_{\mathrm{ss}} = 0$ with $\mathrm{Tr}\,\hat{\rho}_{\mathrm{ss}} = 1$, which the solver computes from the model itself:

```
state = solver.steady_state(check_unique=None)       # DensityState
solver.feed_model(model, initial_state=state)        # or initial_state="steady"
```

`steady_state` solves the linear system

$$\big(\mathcal{L} + |\hat{r}\rangle\!\rangle\langle\!\langle\mathbb{1}|\big)|\hat{\rho}\rangle\!\rangle = |\hat{r}\rangle\!\rangle, \qquad \mathrm{Tr}\,\hat{r} = 1, \tag{24}$$

with $\hat{r} = \mathbb{1}/D_{\mathrm{H}}$ and $\langle\!\langle\mathbb{1}|$ the trace functional. Applying $\langle\!\langle\mathbb{1}|$ to it gives $\mathrm{Tr}\,\hat{\rho} = 1$ because $\langle\!\langle\mathbb{1}|\mathcal{L} = 0$ (trace preservation), hence $\mathcal{L}\hat{\rho} = 0$. The matrix is invertible exactly when the kernel of $\mathcal{L}$ is one-dimensional, and its solution is then the stationary state, whatever $\hat{r}$. The dense backend factorizes the matrix (LU); the sparse backend applies it matrix-free and solves it with GMRES (restart 100, tolerance `krylov_tolerance`), without preconditioner, within 100 restart cycles unless `krylov_maxiter` is set; a stiff generator, whose relaxation gap is small next to $\|\mathcal{L}\|$, can exhaust them, in which case `ConvergenceError` names the remedies.

The state depends only on the generator, not on the reference state of the model or on the thermodynamic context. It is returned as a `DensityState` that contains the sector blocks above 10^{-14}, with the diagnostics in `state.metadata["steady_state"]`: `method`, `relative_residual` ($\|\mathcal{L}\hat{\rho}\|/\|\hat{\rho}\|$), `hermiticity_error` (before symmetrization), `minimum_eigenvalue`, and the evidence on uniqueness. The state of a unique kernel is a density matrix; a `SteadyStateWarning` is emitted when it is not (smallest eigenvalue below -10^{-8}, Hermiticity error above 10^{-8}, or a large residual).

Uniqueness. If the kernel of $\mathcal{L}$ has dimension above one (a closed model, conserved quantities, a level or sector without relaxation, dark states of a collective decay), there is no stationary state but a family, and what the solver returns is one element of it, without physical meaning; a `SteadyStateWarning` says so. The dense backend detects it for free: the reciprocal condition number ρ of the LU gives the error estimate ϵ/ρ, and the kernel is declared degenerate (`unique=False`, minimum-norm solution) above 10^{-6}. The sparse backend does not test it by default (`unique=None`), because the test is as stiff as the generator: with `check_unique=True` it solves a second system with a random $\hat{r}$, which for a degenerate kernel gives another element (GMRES returns the projection

of $\hat{r}$ on the kernel); the solve is limited to 2000 iterations and leaves `unique=None` if it has not converged, as it does for a stiff unique generator from $D_{\mathrm{H}} \gtrsim 30$ with a rate $1/1000$ of the energy scale. For a small model, the dense backend is the tool to ask whether the stationary state is unique. For a closed model the maximally mixed state is returned with a warning (it is stationary, as is any state diagonal in the eigenbasis).

Imposing a reference state. `feed_model(model, context=None, initial_state=None)` accepts a `DensityState` or a `PureState` for `initial_state`, or the string `"steady"`, which computes the stationary state of the generator and takes it. The state replaces the one the model would supply (its `equilibrium_state(context)` or `initial_condition`, which are then not called, so that a thermal context without `energy_unit` is not an error); `context` is still given to the model for its collapse channels. A `DensityState` is validated like any reference state (Hermitian, unit trace, positive); another type raises `TypeError`. The stationarity check then runs as usual and passes, r being of the order of the residual of the solve. For the two-level system of the example above, imposing the stationary state removes the 9% of excited population: the linear response at resonance equals that of the ground state, and for channels that satisfy detailed balance (rates $\gamma_\downarrow$ and $\gamma_\uparrow = \gamma_\downarrow e^{-\hbar\omega/k_B T}$) the stationary state is the Gibbs state, which `steady_state` reproduces to 10^{-10}. With rates $\gamma_\downarrow$ and $\gamma_\uparrow$ that do not, the excited population is $\gamma_\uparrow/(\gamma_\uparrow + \gamma_\downarrow)$ and the linear response scales with the ground population $\gamma_\downarrow/(\gamma_\uparrow + \gamma_\downarrow)$.

2.4.5 Pathway registration

The active pathways are registered with `set_pathways(pathways)`. The input may contain `FrequencyPathway` objects or mappings from which these objects can be constructed. Names must be unique and at least one pathway must be supplied. The method returns the solver instance. Active pathways are queried with `get_pathways(component=None)`.

If `component` is omitted, all active pathways are returned. If a component is provided, only pathways assigned to that component are returned. The aliases `"nonrephasing"` and `"nonrephase"` are normalized to the internal component name `"unrephasing"`.

The method `pathway_summary()` returns serializable pathway metadata without exposing the internal model or backend objects.

2.4.6 Propagation protocols

A `SpectroscopyProtocol` is an ordered tuple of `PropagationInterval` objects. Each interval has a unique name, a domain in `"time"`, `"frequency"`, or `"identity"`, an optional coherence order, an optional interval-specific broadening `eta`, and a flag `signed`. By default the protocol checks the modulus of the coherence order of a pathway at an interval ($+NQ$ and $-NQ$ are both accepted); with `signed=True` it checks the sign as well. The protocol validates pathway length, coherence-order magnitude, and coordinates. It separately exposes all coordinate names and its frequency- and time-axis names.

The helper

```
standard_nq_protocol(
    order,
    nq_interval,
    detection_interval,
```

```
    *,
    n_interactions=3,
    nq_axis=None,
    detection_axis="omega3",
)
```

constructs the common two-frequency-axis protocol, with the remaining propagation intervals kept in the time domain.

2.4.7 Single-pathway calculation

The method `calc_pathway(pathway, protocol, coordinates)` evaluates one pathway at one coordinate point.

The input are:

- `pathway`: a `FrequencyPathway` object or the name of an active pathway;
- `protocol`: a SpectroscopyProtocol describing the propagation intervals;
- `coordinates`: a mapping from interval names to numerical time or frequency values.

The protocol determines which intervals are propagated in the time or frequency domain. Every non-identity interval must have a corresponding entry in `coordinates`. The method return a `PathwayResult` containing

- the complex pathway response;
- the pathway name;
- optional numerical diagnostics;
- a mapping of additional observable values.

If $S_p(\omega)$ denotes the response of pathway p, then the returned value is the complex quantity

$$S_p(\omega) = S_p^{Re}(\omega) + iS_p^{Im}(\omega). \tag{25}$$

2.4.8 Several observables from one pathway propagation

The method

```
calc_pathway_observables(
    pathway,
    protocol,
    coordinates,
    observables=None
)
```

propagates a pathway once and evaluates several readouts from its propagated response. The pathway's default detection is always included; additional names or `ObservableSpec` objects are normalized and deduplicated. Ordinary operator and jump-rate readouts reuse the final response. Action-detected and integrated-jump specifications can require extra post-pathway interaction branches or propagation, as described below.

2.4.9 Component calculation

The method `calc_component(...)` sums selected pathways belonging to one component. If $\mathcal{P}_C$ denotes that set, the returned response is

$$S_C(\omega) = \sum_{p \in \mathcal{P}_C} S_p(\omega). \tag{26}$$

If `pathways=None`, all pathways belonging to the requested component are used; otherwise only the supplied subset is included.

2.4.10 Multidimensional spectrum generation

The method

```
generate_spectrum(
    protocol,
    axes,
    fixed_coordinates=None,
    pathways=None,
    observables=None,
    warn_null=True,
    strategy="auto",
    memory_limit_mb=1024.0,
)
```

evaluates the selected pathways over a multidimensional coordinate grid. The argument `axes` is a non-empty ordered mapping

```
coordinate_name:  one_dimensional_values
```

where each coordinate name must correspond to a non-identity interval in the protocol. Coordinates not included in axes must be supplied through `fixed_coordinates`.
For example, a two-dimensional spectrum can be requested using

```
axes = {
    "omega1": omega1_values,
    "omega3": omega3_values,
}
fixed_coordinates = {
    "t2": waiting_time,
}
```

The resulting array shape follows the insertion order of `axes`. The returned `SpectrumResult` contains

- `axis_names`, `axis_values`, and `fixed_coordinates`;
- `pathways[name]` and component sums in `components[name]`;
- per-pathway arrays in `observables`, indexed first by observable and then by pathway;

- component sums in `observable_components`, indexed first by observable and then by component;
- pathway metadata and numerical diagnostics, including `diagnostics["null_pathways"]` (Sec. 2.6.6) and the strategy and work counters of Sec. 2.4.11.

The default detection of each pathway remains in the legacy `pathways` and `components` mappings. Additional requested observables are stored separately, preventing one detector from overwriting another.
The component arrays are the sums of the corresponding pathway arrays,

$$S_C(\boldsymbol{\xi}) = \sum_{p \in C} S_p(\boldsymbol{\xi}), \tag{27}$$

where $S_p(\boldsymbol{\xi})$ is evaluated at every grid point $\boldsymbol{\xi}$ by one of the two strategies below.

2.4.11 Evaluation strategies

A pathway response is the functional $\langle\langle d| \mathcal{O}_K \cdots \mathcal{O}_1 |\hat{\rho}_0\rangle\rangle$ of an ordered chain of operations $\mathcal{O}_k$: the interaction superoperators, the propagators $\exp(\mathcal{L}t)$ of the time intervals and the resolvents $\mathcal{R}_\eta(\omega)$ of the frequency intervals; the detection vector d is $\mathrm{vec}(\hat{O}^\dagger)$ for an operator observable, and includes the readout interaction of an action-detected observable and the integrated detector of a jump count. The argument `strategy` selects how the grid is evaluated.

`"pointwise"` The chain is propagated at every grid point. Two scanned frequency axes of N_1 and N_2 points cost $2N_1N_2$ shifted solves per pathway; additional observables reuse the propagation.

`"factorized"` The chain is cut between two scanned intervals, at an operation c chosen to minimize the number of solves. With $\boldsymbol{\xi}_L$ the points of the axes before the cut and $\boldsymbol{\xi}_R$ those after it,

$$S_p(\boldsymbol{\xi}_L, \boldsymbol{\xi}_R) = c_p \, \langle\langle G(\boldsymbol{\xi}_R) \,|\, F(\boldsymbol{\xi}_L) \rangle\rangle, \quad |F\rangle\rangle = \mathcal{O}_{c-1} \cdots \mathcal{O}_1 |\hat{\rho}_0\rangle\rangle, \quad |G\rangle\rangle = (\mathcal{O}_K \cdots \mathcal{O}_c)^\dagger |d\rangle\rangle. \tag{28}$$

Each side is evaluated once per point of its own axes and the grid is obtained from one matrix product. The adjoint of a resolvent is $\mathcal{R}_\eta(\omega)^\dagger = [(\eta + i\omega)\mathbb{1} - \mathcal{L}^\dagger]^{-1}$, that of a propagator is $\exp(\mathcal{L}^\dagger t)$, and that of a ket (bra) interaction $\hat{\rho} \mapsto T\hat{\rho}$ ($\hat{\rho} \mapsto \hat{\rho}T^\dagger$) is $\hat{Y} \mapsto T^\dagger \hat{Y}$ ($\hat{Y} \mapsto \hat{Y}T$). For two axes the cost is $N_1 + N_2$ solves per pathway and observable instead of $2N_1N_2$. Pathways that begin with the same operations share them (the three rephasing pathways of the open two-level system start with the same bra interaction and the same resolvent, so they share the whole forward part), the detection vectors of several observables are carried as the columns of one adjoint chain, and operations at fixed coordinates between the two scanned intervals are applied on the side that has fewer vectors. It requires at least two scanned axes; with three axes the cut is placed where it costs least.

`"auto"` (default) takes the factorized evaluation when it applies, needs fewer solves than the pointwise one and fits in the memory limit, and the pointwise evaluation otherwise.

The two strategies give the same numbers up to the accuracy of the linear solves: about 10^{-15} of the largest value on the dense backend and the GMRES tolerance (`krylov_tolerance`, 10^{-10} by default) on the sparse backend. The pointwise evaluation is kept for validation and for the cases in which the factorized one does not apply: a single scanned axis, and the `SparseSectorBackend` for a closed model with a pure state and time intervals only, which is evaluated as a rank-one dyad

of two Hilbert-space vectors, cheaper than any Liouville vector. Requesting `"factorized"` when it does not apply raises `ValueError`.

The factorized evaluation stores the vectors of the side with fewer vectors for the whole grid (at most N vectors of $D_{\mathrm{L}} = D_{\mathrm{H}}^2$ complex entries for each distinct chain) and streams the other side in slabs of its first axis. `memory_limit_mb` bounds the vectors in memory, the slab size being reduced until they fit; the number of solves does not change. If even one slab does not fit, `"auto"` falls back to the pointwise evaluation, and `"factorized"` proceeds with slabs of one index.

The diagnostics of the result report what was done:

- `diagnostics["strategy"]`: `"pointwise"` or `"factorized"`;
- `"resolvent_solves"`: resolvents applied to a vector (one GMRES run on the sparse backend; one solve or product with a cached inverse on the dense backend);
- `"resolvent_factorizations"`: dense matrices factorized or inverted (dense backend; the factorized evaluation factorizes each shifted matrix once for all its vectors and does not keep the inverse);
- `"propagations"`: time propagators applied to a vector, cache hits excluded;
- `"maximum_resolvent_residual"`: largest absolute residual of the GMRES solves (sparse backend);
- `"solver"`: iterations, residuals and conditioning of the linear solves (Sec. 2.4.18);
- `"factorization"` (factorized evaluation only): the cut, the axes of each side, the side that is stored, the number of vectors stored, the number of slabs and the number of detection vectors.

In the pointwise evaluation `diagnostics["pathways"][name]` holds one record per grid point; in the factorized evaluation it is empty. The counters can be read at any time with `solver.backend.counters()` and reset with `reset_counters()`.

2.4.12 Coherence-order-resolved-spectrum

The method

```
generate_nq_spectrum(
    order,
    protocol,
    axes,
    fixed_coordinates=None,
    pathways=None,
    observables=None,
    warn_null=True,
    strategy="auto",
    memory_limit_mb=1024.0
)
```

generates a spectrum while separating pathways according to their $+NQ$ and $-NQ$ coherence-order contributions. The requested order must correspond to exactly one protocol interval whose declared coherence order has absolute value N. For $N > 0$, the returned components include

$$+NQ, \qquad -NQ, \qquad NQ = (+NQ) + (-NQ). \tag{29}$$

For $N = 0$, the total contribution is stored as

$$0Q. \tag{30}$$

The same coherence-order separation is applied to the observable-component mapping. Individual pathway and observable arrays remain available in the returned result.

2.4.13 Result containers

`PathwayResult` stores a scalar complex `value`, the corresponding pathway, diagnostics, and observable values. `SpectrumResult` stores the multidimensional arrays described above and exposes their common `shape`. Plotting methods return a `PlotResult` containing the figure, axes, panel names, selected data, and effective configuration. These containers separate numerical results from rendering and make all aggregation steps inspectable.

2.4.14 Jump-channel access

The method `jump_channel_names()` returns the GKSL channel names supplied by the model. These names are used by instantaneous or integrated jump observables.

2.4.15 Decay-rate diagnostic

The rate of each channel is the Lindblad coefficient γ_c; its relation to measured decay times depends on the normalization of $\hat{L}_c$. The method

```
rates = solver.decay_rates(modes=False, max_mode_dimension=40)
```

diagonalizes the Hamiltonian and reports, in its eigenbasis $\{|x\rangle\}$, the rates implied by the declared channels, read from the diagonal of the generator in the basis of dyads $|x\rangle\langle y|$:

$$\Gamma_x = \sum_c \gamma_c \left(\langle x| \hat{L}_c^\dagger \hat{L}_c |x\rangle - |\langle x| \hat{L}_c |x\rangle|^2 \right), \qquad W_{y \leftarrow x} = \sum_c \gamma_c |\langle y| \hat{L}_c |x\rangle|^2, \tag{31}$$

$$\gamma_{xy} = \sum_c \gamma_c \left[\tfrac{1}{2} \langle x| \hat{L}_c^\dagger \hat{L}_c |x\rangle + \tfrac{1}{2} \langle y| \hat{L}_c^\dagger \hat{L}_c |y\rangle - \mathrm{Re}\left(\langle x| \hat{L}_c |x\rangle \langle y| \hat{L}_c |y\rangle^* \right) \right]. \tag{32}$$

The returned `DecayRates` object holds the energies, the dominant sector of each eigenstate, the population decay rates Γ_x (`population_decay`), the transfer rates $W_{y \leftarrow x}$ (`transition_rates`), and the coherence decay rates and frequencies γ_{xy}, ω_{xy} (`coherence_decay`, `coherence_frequency`). The method `coherences(between=(sector_a, sector_b))` lists the coherences between two manifolds, for example the optical coherences between the ground and one-excitation manifolds. These secular rates are the pole half-widths of frequency-resolved intervals when the generator does not couple different dyads. With `modes=True`, the exact eigenvalues $\lambda_\nu = -\kappa_\nu - i\Omega_\nu$ of the generator are added for $D \leq$ `max_mode_dimension`. For the open two-level system of the Quick start, `decay_rates()` returns γ_1 for the excited-state population and $\Gamma_2 = \gamma_1/2 + \gamma_\phi$ for the optical coherence; a $\hat{\sigma}_z$ channel given the rate γ_ϕ instead of $\gamma_\phi/2$ would appear as $\gamma_1/2 + 2\gamma_\phi$. If no dissipative channels are present, the returned tuple is empty.

2.4.16 Direct Hilbert-space resolvent

The public method `solve_hilbert_resolvent(...)` delegates the shifted Hilbert-space solve to the active backend. It is available only when the backend advertises this capability; otherwise the solver raises a capability error. The dense backend uses a direct solve and the sparse backend uses GMRES with convergence diagnostics.

2.4.17 Memory estimation

The method `estimate_memory()` returns a structural memory estimate for the active backend. The result includes the total Hilbert-space dimension and the memory required for one complex state vector. For the dense Liouville backend, it additionally reports the Liouville-space dimension and the estimated memory for one density vector and one dense Liouville matrix:

$$D_L = D_H^2 \tag{33}$$

where D_H is the total Hilbert-space dimension. For the sparse backend, the estimate is structural rather than an exact prediction of the final allocation.

2.4.18 Numerical backend architecture

The numerical backend is responsible for assembling the effective generator and propagating the pathway response in the time or frequency domain. The public `SpectroscopySolver` orchestrates the calculation, while the backend determines how the Hilbert- and Liouville-space operations are represented numerically.

Let

$$D_{\mathrm{H}} = \sum_{s \in \mathcal{S}} d_s \tag{34}$$

denote the total Hilbert-space dimension. The corresponding vectorized Liouville space has dimension

$$D_{\mathrm{L}} = D_{\mathrm{H}}^2. \tag{35}$$

Common generator

Both released backends use the same effective generator,

$$\mathcal{L}[\hat{\rho}] = -i[\hat{H}, \hat{\rho}] + \sum_c \gamma_c \left(\hat{L}_c \hat{\rho} \hat{L}_c^\dagger - \frac{1}{2} \left\{ \hat{L}_c^\dagger \hat{L}_c, \hat{\rho} \right\} \right). \tag{36}$$

The Hamiltonian and collapse operators are assembled from the sector-resolved blocks supplied by the external `SectorModel`. Inter-sector blocks are allowed. The backend therefore does not assume that the global Hamiltonian is block diagonal.

The common internal generator is represented as a matrix-free linear action whenever possible:

$$\mathrm{vec}(\mathcal{L}[\hat{\rho}]) = \mathcal{L}\,\mathrm{vec}(\hat{\rho}), \tag{37}$$

where the matrix of $\mathcal{L}$ acting on the vectorized density operator carries the same symbol. The released backends use the convention

$$\mathcal{L} = -i[\hat{H}, \cdot\,] + \sum_c \gamma_c \mathcal{D}_c. \tag{38}$$

The action on a density matrix is evaluated with products of the sector blocks with the whole matrix $\hat{\rho}$, not column by column: $\hat{H}\hat{\rho}$ and $\hat{\rho}\hat{H}$ for the Hamiltonian, and five products per channel ($\hat{L}_c\hat{\rho}$, $\hat{\rho}\hat{L}_c^\dagger$, $\hat{L}_c\hat{\rho}\hat{L}_c^\dagger$, $\hat{L}_c^\dagger\hat{L}_c\hat{\rho}$ and $\hat{\rho}\hat{L}_c^\dagger\hat{L}_c$, the first two being reused). One application of $\mathcal{L}$ therefore costs $2 + 5n_c$ block-by-matrix products for n_c channels.

A frequency-domain interval with frequency ω and broadening η solves the shifted Liouville equation

$$[(\eta - i\omega)\mathbb{1} - \mathcal{L}]\, x = \mathrm{vec}(\hat{\rho}). \tag{39}$$

The parameter η is a numerical or phenomenological homogeneous broadening and is distinct from the detection efficiency used by jump observables.

DenseLiouvilleBackend

`DenseLiouvilleBackend` is the reference backend for small Hilbert spaces. During construction, it materializes the global Hamiltonian and the complete Liouville generator:

$$\hat{H} \in \mathbb{C}^{D_{\mathrm{H}} \times D_{\mathrm{H}}}, \qquad \mathcal{L} \in \mathbb{C}^{D_{\mathrm{L}} \times D_{\mathrm{L}}}. \tag{40}$$

The time-domain propagator is explicitly constructed as

$$\mathcal{U}(t) = \exp(\mathcal{L}t), \tag{41}$$

using a dense matrix exponential. A frequency-domain interval uses the dense inverse

$$\mathcal{R}_\eta(\omega) = [(\eta - i\omega)\mathbb{1} - \mathcal{L}]^{-1}. \tag{42}$$

The dense backend also provides a Hilbert-space resolvent,

$$R_{\mathrm{H}}(\omega, \eta) = \left[(\omega + i\eta)\mathbb{1} - \hat{H}\right]^{-1}, \tag{43}$$

which is solved using a dense linear-system solver. This Hilbert-space resolvent follows the retarded convention; for the Hamiltonian evolution of a ket it is related to the Liouville-space resolvent of the main text by $\mathcal{R}_\eta = i\, R_{\mathrm{H}}$.

Dense propagation is useful for:

- small two-level or few-level systems;
- validating another backend;
- checking pathway and phase conventions;
- comparing against explicitly constructed Liouville matrices.

Its main limitation is memory scaling. A dense Liouville matrix contains $D_{\mathrm{L}}^2 = D_{\mathrm{H}}^4$ complex entries. It therefore becomes impractical rapidly as the Hilbert-space dimension increases.

The dense backend optionally caches time propagators and resolvents. Cache sizes are bounded by the `max_cache_entries` option. The factorized evaluation of a grid (Sec. 2.4.11) solves each shifted system by one factorization for all the vectors that need that frequency, applies the adjoint resolvent as the solution of the conjugate-transposed system, and does not store the inverses: unlike the pointwise evaluation, it does not depend on the cache holding every frequency of the grid.

SparseSectorBackend

`SparseSectorBackend` avoids constructing a global dense Hamiltonian or a dense $D_{\mathrm{L}} \times D_{\mathrm{L}}$ Liouville matrix. Instead, it keeps the sector-resolved operators as linear actions.

For a Hilbert-space vector v, the Hamiltonian action is assembled from the sector blocks,

$$(Hv)_t = \sum_s \hat{H}_{ts} v_s, \tag{44}$$

where $\hat{H}_{ts}$ is the block mapping source sector s to target sector t. This allows inter-sector couplings while preserving the block structure supplied by the model.

For time propagation, the backend uses Krylov-based actions of the form

$$v(t) = \exp\Big(-i\hat{H}t\Big)v(0). \tag{45}$$

For a pure initial state and unitary time-domain pathway, the ket and bra branches can be propagated separately. The response is then represented as a rank-one dyad rather than as a full D_{L}-component density vector.

This representation is exact for a unitary evolution of a pure initial state, up to the numerical accuracy of the Krylov propagation.

The backend switches to a Liouville-space representation when at least one of the following conditions holds:

- the protocol contains a frequency-domain interval;
- the generator is dissipative;
- the initial state is mixed.

In this case, the density response is stored as a vector of size D_{L}, but the Liouville generator is still applied through a matrix-free action. Time propagation uses a Krylov exponential action,

$$\mathrm{vec}(\hat{\rho}(t)) = \exp(\mathcal{L}t)\,\mathrm{vec}(\hat{\rho}(0)), \tag{46}$$

without constructing the matrix of $\mathcal{L}$ explicitly.

A frequency-domain interval is evaluated by solving

$$[(\eta - i\omega)\mathbb{1} - \mathcal{L}]\,x = \mathrm{vec}(\hat{\rho}) \tag{47}$$

with the GMRES iterative solver. The iterations and the residual of every solve are recorded (see the paragraph on the accuracy of the solves below). If GMRES does not converge within the requested tolerance, the backend raises a convergence error.

The sparse backend also exposes a Hilbert-space resolvent,

$$\Big[(\omega + i\eta)\mathbb{1} - \hat{H}\Big]\,x = v, \tag{48}$$

which is solved iteratively with GMRES.

The sparse backend therefore avoids the largest dense allocation, but it does not eliminate the D_{H}^2 Liouville vector required for frequency-domain, mixed-state, or dissipative calculations.

Options of the sparse backend. The options below are passed to `SpectroscopySolver` together with `backend="sparse_sector"` and the broadening `eta`.

- `krylov_tolerance` (default 10^{-10}): relative residual required from each GMRES solve;
- `krylov_maxiter` (default `None`): maximum number of GMRES iterations; `None` leaves the choice to SciPy;
- `resolvent_restart` (default `None`, that is the SciPy default of 20): restart length of GMRES, see below;
- `preconditioner` (default `"auto"`): `"auto"`, `"diagonal"` or `None`, see below; any other value raises a `ValueError`;
- `diagonal_solve` (default `True`): solve the shifted systems by division when the generator is exactly diagonal, see below;
- `cache_propagations` (default `True`) and `max_cache_entries` (default 32): caching of propagated states and its bound;
- `record_solves` (default `False`, also accepted by the dense backend): keep one record per linear solve in `diagnostics["solver"]["records"]` (Sec. 2.4.18).

Accuracy of the solves. `result.diagnostics["solver"]` summarizes the shifted solves of a call to `generate_spectrum`:

- `"method"` (`"gmres"`, `"direct"` or `"diagonal"`) and `"solves"`;
- `"iterations_mean"` and `"iterations_max"` (GMRES); a division-solved generator reports `"exact": True` instead, with no iteration and no residual;
- `"relative_residual_max"` and `"relative_residual_mean"`: $\|b - Sx\|/\|b\|$ for the shifted system $Sx = b$, $S = (\eta - i\omega)\mathbb{1} - \mathcal{L}$;
- `"condition_estimate"` and `"forward_error_estimate"`, below;
- `"records"`, one dictionary per solve (`omega`, `eta`, `adjoint`, `iterations`, `relative_residual`), when the backend option `record_solves=True` is given.

The residual is the true one, not the preconditioned one on which GMRES stops, and it is a backward error: x solves $(S + E)x = b$ exactly with $\|E\| \le \|b - Sx\|/\|x\|$. The relative error of x is at most $\kappa(S) = \|S\|\|S^{-1}\|$ times the relative residual. For a normal generator, such as a closed model in its eigenbasis, the eigenvalues of S are $\eta + \kappa_\nu - i(\omega - \Omega_\nu)$ with $\kappa_\nu \ge 0$, so $\|S^{-1}\| \le 1/\eta$, $\|S\| \le \eta + |\omega| + \|\mathcal{L}\|$ and

$$\kappa(S) \le \frac{\eta + |\omega| + \|\mathcal{L}\|}{\eta} \simeq \frac{\|\mathcal{L}\|}{\eta}, \tag{49}$$

whatever the dimension. For a dissipative generator the same resolvent has trace-norm norm at most $1/\eta$, because $\exp(\mathcal{L}t)$ is a completely positive trace-preserving map and the resolvent is its Laplace transform; in the Hilbert–Schmidt norm of the solver this gives $\|S^{-1}\| \le \sqrt{D_{\mathrm{H}}}/\eta$, a worst case that the examples do not approach. `"condition_estimate"` is the right-hand side above, with $\|\mathcal{L}\|$ obtained by power iteration (`backend.generator_norm()`, a lower estimate) and the maximum over the frequency intervals of the protocol; it is `None` when no interval is in the frequency domain.

`"forward_error_estimate"` is its product with the largest relative residual. In the three examples of the article the error of a solve is of the order of its relative residual, far below this bound. The Hilbert-space resolvent returns `iterations` and `relative_residual` in its diagnostics. The dense backend reports the residual of its direct solves in the factorized evaluation; the pointwise evaluation applies cached inverses and reports none.

Reducing the residual is a matter of `krylov_tolerance`. Iterative refinement, which solves $S\,\delta = b - Sx$ and updates x, was evaluated and is not offered: each step costs as many GMRES iterations as the solve it corrects, so that reaching 5×10^{-13} from 10^{-6} takes 36 iterations, summed over four frequencies, on the thermal four-site chain at $\eta = 0.03$ meV, whereas a solve with `krylov_tolerance=1e-12` takes 28 for an error of 9×10^{-16}.

Diagonal preconditioner. With `preconditioner="diagonal"` (and with the default `"auto"` when the Hamiltonian is diagonal, see below), each GMRES solve of the Liouville resolvent is preconditioned by the inverse of the shifted diagonal of the generator,

$$M_\omega^{-1} = \mathrm{diag}[(\eta - i\omega)\mathbb{1} - \mathcal{L}]^{-1}\,, \tag{50}$$

and each solve of the Hilbert-space resolvent by the inverse of the shifted diagonal of $\hat{H}$. The diagonal of $\mathcal{L}$ is obtained from the sector blocks without forming the matrix. The model adapters express the operators in the eigenbasis of the Hamiltonian (sector by sector for `ExcitationSectorModel`). The generator of a closed model, one without collapse channels, is then diagonal, and the preconditioner is exact: every GMRES solve converges in one iteration, for any η and any dimension (with the default `diagonal_solve=True` such a solve is done by division instead, see below). It stays exact when every collapse operator is diagonal in that basis, as for pure dephasing. With collapse channels that couple coherences the diagonal is only an approximation: the number of iterations decreases but still depends on η. The preconditioner does not change the result beyond the solver tolerance.

The default `preconditioner="auto"` applies the diagonal preconditioner when every Hamiltonian block is a diagonal block of a single sector (a model in its eigenbasis, which is what the adapters provide, without coupling between sectors), whatever the collapse operators, and none otherwise. Blocks that are arrays or sparse matrices are inspected (off-diagonal entries below 10^{-12} of the largest entry count as zero); matrix-free blocks cannot be, and the preconditioner is then left off, as it is for a model given in a basis that does not diagonalize its Hamiltonian. The choice made is stored in `backend.preconditioner` and reported as `diagnostics["solver"]["preconditioner"]`. `None` always gives plain GMRES and `"diagonal"` forces the preconditioner, which is valid for any model.

The preconditioner is exact when the generator is diagonal, a closed model or diagonal collapse operators; otherwise the collapse operators perturb the diagonal, which keeps the energy differences, and it remains an approximation that is nevertheless far better than none. This was checked against plain GMRES (`krylov_tolerance=1e-10`, restart 20, three kinds of start vector including a random one) on the three-site chain with dissipation scaled from 0.1 to 1000 and on the six-site XXZ chain of Example 2 ($D_\mathrm{H} = 57$, 4 K) with collective decay or local dephasing from 10^{-3} to 10 meV, at two broadenings each (`bench_preconditioner_dissipative.py`). The ratio of the mean number of iterations (preconditioned over plain) lies between 3×10^{-4} and 0.76 in 28 of the 30 cases, with a plain GMRES that reaches 54 000 iterations in one solve against 1 114 preconditioned; the true residuals are comparable and below 10^{-10}. Only a local dephasing rate of 10 meV, an order of magnitude above the energy scale of the chain and far from any use in Example 2, is slightly unfavorable: 1.20 and 1.23 (about 40 iterations against 33).

Exact diagonal solve. When the generator is exactly diagonal in the dyad basis $|x\rangle\langle y|$, which holds when every Hamiltonian block is a diagonal block of one sector (a model in its eigenbasis) and every collapse operator is a diagonal block too (none, or pure dephasing, possibly with complex diagonal entries), $\mathcal{L}|x\rangle\langle y| = \mathcal{L}_{xy}|x\rangle\langle y|$ with

$$\mathcal{L}_{xy} = -i(E_x - E_y) + \sum_c \gamma_c \Big[L_{c,xx} L^*_{c,yy} - \tfrac{1}{2}\big(K_{c,xx} + K_{c,yy}\big) \Big], \qquad K_c = L_c^\dagger L_c, \tag{51}$$

and `diagonal_solve=True` (the default) solves every shifted system, direct or adjoint, by the element-wise division $x = b/[(\eta - i\omega) - \mathcal{L}_{xy}]$, and the Hilbert-space resolvent of a diagonal Hamiltonian by $x = b/[(\omega + i\eta) - E_x]$. This is the sum over states: exact to rounding, one pass over the vector, no iteration, whatever η and the dimension. The real part of $\mathcal{L}_{xy}$ is not positive, so the denominator does not vanish for $\eta > 0$; for $\eta = 0$ on a pole the solver raises a `ConvergenceError` instead of dividing by zero. The backend attribute `generator_is_diagonal` says whether the generator qualifies; a collapse operator that moves population ($|j\rangle\langle i|$) or couples coherences, or a block that cannot be inspected (matrix-free), makes it false, and GMRES with the preconditioner above runs. `diagonal_solve=False` keeps GMRES in every case, which is how the preconditioner is tested; the dense backend always inverts the matrix and serves as the independent reference.

Restart of GMRES. `resolvent_restart=None` leaves the restart length to SciPy ($\min(20, n)$). A longer restart reduces the number of iterations but makes each of them dearer and stores $m + 1$ Krylov vectors of $D_\mathrm{L} = D_\mathrm{H}^2$ entries (85 MB for $m = 200$ at $D_\mathrm{H} = 163$). No value wins on the problems measured, so the default is kept. Unpreconditioned solves of the thermal spin chain of Example 2 (`preconditioner=None`) took, per solve, with $m = 20, 50, 100, 200$:

	$m = 20$	$m = 50$	$m = 100$	$m = 200$
$D_\mathrm{H} = 57$, $\eta = 0.03$ meV: iterations (time)	673 (0.12 s)	321 (0.09 s)	134 (0.06 s)	102 (0.06 s)
$D_\mathrm{H} = 57$, $\eta = 0.003$ meV	5688 (0.99 s)	3539 (0.98 s)	1895 (0.87 s)	128 (0.08 s)
$D_\mathrm{H} = 163$, $\eta = 0.03$ meV	1210 (1.4 s)	1081 (1.7 s)	939 (2.6 s)	798 (4.1 s)

so that a long restart is worth ten at the smallest η and costs a factor three at the largest dimension. The solves of Examples 1 and 3 and of the dissipative three-site chain take two or three iterations whatever the restart. With the preconditioner, which is the default whenever it is exact, the restart is immaterial for the models of the article.

```
solver = SpectroscopySolver(
    backend="sparse_sector",
    eta=0.03,
    krylov_tolerance=1e-11,
    preconditioner="diagonal",
)
solver.feed_model(model, context=thermal_context)
```

For the thermal spin chain of Example 2 ($D = 57$, $\eta = 0.03$ meV), the mean number of GMRES iterations per solve falls from about 640 to 1, and the time for a 3×3 subgrid from 58 s to 0.4 s. The measurements as a function of D and η are given in Sec. 6 of the article and in `validation/benchmarks/example2_benchmark.ipynb`.

Both released backends are intended to represent the same physical generator. The difference is numerical representation and linear-algebra strategy, not a different physical model. The dense backend is useful as a reference implementation, while the sparse backend is the preferred default for sector-resolved calculations.

Backend selection

The solver accepts the following backend names:

- `"dense"` or `"dense_liouville"`: `DenseLiouvilleBackend`;
- `"sparse"` or `"sparse_sector"`: `SparseSectorBackend`;
- `"auto"`: automatic selection among the released backends.

The default is

```
backend="sparse_sector"
```

When `backend="auto"` is selected, the solver compares the model requirements with the declared backend capabilities. The requirements include:

- pure or mixed initial-state support;
- unitary or Lindblad generator;
- time- or frequency-domain support;
- exactness requirements;
- matrix-free requirements.

The built-in automatic selection currently considers the released sparse and dense backends. A custom backend registered through `register_backend` can be selected explicitly by name.

2.4.19 Solver Summary

The method `summary()` returns a serializable description of the constructed calculation without exposing the underlying external model. The summary includes

- the selected backend;
- backend capabilities and support;
- model requirements and capabilities;
- the sector labels and dimensions;
- the total Hilbert-space dimension;
- the stationarity residual of the reference state;
- the active pathway metadata;
- the transition-operator decomposition and its `transition_summary()` when they are advertised by the model.

This method is intended for inspection, reproducibility records, and debugging of backend or pathway configuration.

2.5 `ObservableSpec`: operator, jump, and action detection

`ObservableSpec` separates the measured quantity from pathway propagation:

```
ObservableSpec(
    name,
    kind="operator",
    operator=None,
    channel=None,
    efficiency=1.0,
    time_window=None,
    n_steps=101,
    fourth_interaction=None,
    integration="exact",
)
```

The efficiency must lie in $[0, 1]$. Integrated observables require $0 \le t_0 \le t_f$; `integration` is `"exact"` (default) or `"trapezoid"`, and `n_steps`, at least two, is used only by the trapezoidal option.

2.5.1 Named operator observables

For `kind="operator"`, `operator` identifies a named model-side observable $\hat{O}$. If it is omitted, the observable name is used. For the final pathway response $\hat{\rho}_p$,

$$S_{O,p} = c_p \operatorname{Tr}(\hat{O}\hat{\rho}_p), \tag{52}$$

where c_p is the pathway response prefactor defined in Section 2.6. Multiple named operators may be exposed by either provided model adapter.

2.5.2 Instantaneous and integrated jump observables

For `kind="jump_rate"`, `channel` identifies a GKSL channel. The pathway-resolved detected rate is

$$I_{c,p}(t) = c_p \varepsilon_{\mathrm{det}} \gamma_c \operatorname{Tr}\Big(\hat{L}_c^\dagger \hat{L}_c \hat{\rho}_p(t)\Big). \tag{53}$$

The channel rate γ_c remains separate from $\hat{L}_c$. Hence the channel must be one of `solver.jump_channel_names()`. A pathway contribution can be complex because it includes c_p, even though the rate of a physical density matrix is real and non-negative.

For `kind="integrated_jump"`, the detected rate is integrated over a detection window that starts after the final pathway interval,

$$N_{c,p} = \int_{t_0}^{t_f} I_{c,p}(t)\, dt = c_p \,\langle\!\langle \hat{X}_c | \hat{\rho}_p \rangle\!\rangle, \qquad \hat{X}_c = \varepsilon_{\mathrm{det}} \gamma_c \int_{t_0}^{t_f} \mathcal{U}^\dagger(t) \big[\hat{L}_c^\dagger \hat{L}_c\big]\, dt. \tag{54}$$

With the default `integration="exact"`, the integrated detector $\hat{X}_c$ is computed once per channel and window, without quadrature, as the last column of the exponential of the augmented generator $\left(\begin{smallmatrix} \mathcal{L}^\dagger & \vec{v} \\ 0 & 0 \end{smallmatrix}\right)$ with $\vec{v} = \mathrm{vec}(\gamma_c \hat{L}_c^\dagger \hat{L}_c)$, followed by $\exp\big(\mathcal{L}^\dagger t_0\big)$ when $t_0 > 0$. The dense backend uses a dense exponential; the sparse backend applies the augmented generator matrix-free with `expm_multiply`. The result is cached, so that each pathway state and grid point costs one inner product. With `integration="trapezoid"`, the propagated response is instead sampled at `n_steps` uniformly spaced times and integrated with the trapezoidal rule; this legacy option has an error of order $(\Delta t)^2$ and a cost proportional to `n_steps` per grid point. The convenience constructor

```
ObservableSpec.mean_jump(
    name,
    channel,
    *,
    time_window=(t0, tf),
    efficiency=1.0,
    n_steps=101,
    fourth_interaction=None,
    integration="exact",
)
```

creates this specification. `observable.instantaneous()` returns the corresponding instantaneous jump-rate detector.

Infinite window. `time_window=(t0, numpy.inf)` integrates the count to infinity. The count rate of a state tends to $\mathrm{Tr}(M\hat{\rho}_{\mathrm{ss}})\,\mathrm{Tr}\hat{\rho}$ with $M = \varepsilon_{\mathrm{det}}\gamma_c\hat{L}_c^\dagger\hat{L}_c$, so the solver integrates the excess over this stationary background, $\hat{X}_{c,[t_0,\infty)} = \int_{t_0}^{\infty}\left[\mathcal{U}^\dagger(s) - \mathcal{P}^\dagger\right]\vec{v}\,ds$ with $\vec{v} = \mathrm{vec}(M^\dagger)$ and $\mathcal{P}^\dagger\vec{v} = \overline{\mathrm{Tr}(M\hat{\rho}_{\mathrm{ss}})}\,\mathrm{vec}(\mathbb{1})$. It equals $\exp\left(t_0\mathcal{L}^\dagger\right)\vec{x}$, where $\vec{x}$ solves $(\mathcal{L}^\dagger - |\mathbb{1}\rangle\!\rangle\langle\!\langle\hat{\rho}_{\mathrm{ss}}|)\,\vec{x} = -(\vec{v} - \mathcal{P}^\dagger\vec{v})$: one dense LU (dense backend) or one GMRES solve (sparse backend, as stiff as the relaxation gap of the generator, so `krylov_maxiter` may need raising), cached per channel and window. When the stationary state has no jumps (zero temperature, decay to the ground state) the background vanishes and $\langle\!\langle\hat{X}|\hat{\rho}\rangle\!\rangle$ is the total count of the state, its quantum yield: $\gamma_r/(\gamma_r + \gamma_{nr})$ for a counted channel r and a non-counted channel nr. The window needs a unique stationary state (`solver.steady_state()` reports it): a degenerate kernel raises a `ConvergenceError` on the dense backend, and the sparse solve may fail to converge; `integration="trapezoid"` with an infinite window raises a `ValueError`.

2.5.3 Action detection through a fourth interaction

The field `fourth_interaction` requests one or more additional perturbative interactions after the final pathway interval and before detection. It accepts an `Interaction`, a pathway label such as `"Ku"`, a mapping, or a sequence of alternatives. The convenience constructor is

```
ObservableSpec.action(
    name,
    fourth_interaction,
    *,
    kind="operator",
    operator=None,
    channel=None,
    efficiency=1.0,
    time_window=None,
    n_steps=101,
    integration="exact",
)
```

For alternatives $a \in \mathcal{I}_{\mathrm{rd}}$, the solver applies each interaction to the propagated response and sums the branches before the requested detector,

$$S^{(4)}_{O,p} = c_p \sum_{a\in\mathcal{I}_{\mathrm{rd}}} c_{\mathrm{rd},a}\,\mathrm{Tr}\Big[\hat{O}\,\mathcal{V}_a(\hat{\rho}_p)\Big], \qquad c_{\mathrm{rd},a} = \begin{cases} +i, & \text{ket action,} \\ -i, & \text{bra action.} \end{cases} \tag{55}$$

This fourth interaction adds no protocol interval or spectral axis. For an integrated jump observable, every projected branch is contracted with the same integrated detector (or, with the trapezoidal option, propagated across the detection window before integration). Dedicated properties expose the normalized projection and whether action detection is active; `without_projection()` returns the underlying detector specification.

2.5.4 Normalization and aliases

`normalize_observables(...)` accepts a string or specification, a sequence, or a mapping from aliases to specifications. It returns a validated tuple with unique names; a later duplicate replaces the earlier specification without changing its original position. The aliases *observable* and *trace* map to the operator kind. The aliases *mean jump* and *integrated rate* map to the integrated-jump kind. At least one effective observable is required. The solver retains the pathway's default detection and appends requested observables to the structured result mappings.

2.6 Pathway conventions

A pathway is an ordered sequence of light–matter interactions acting on the ket or bra branch of the Liouville-space density operator. In QuDPy-FDGF, each interaction is represented by an `Interaction` object and each complete pathway by a `FrequencyPathway` object.

2.6.1 Interaction fields

An interaction is defined by

```
Interaction(
    label,
    operator="light_matter",
    pulse_index=None,
    source_sector=None,
    target_sector=None,
)
```

The fields have the following meanings:

- `label` identifies the side and transition direction;
- `operator` gives the named transition operator;
- `pulse_index` identifies the associated pulse;
- `source_sector` optionally constrains the source sector;
- `target_sector` optionally constrains the target sector.

The side of an interaction is determined by its label:

$$\texttt{Ku}, \texttt{Kd} \;\Rightarrow\; \text{ket}, \qquad \texttt{Bu}, \texttt{Bd} \;\Rightarrow\; \text{bra}. \tag{56}$$

The transition direction applied internally is

$$\texttt{Ku}, \texttt{Bu} \;\Rightarrow\; \hat{\mu}_+, \qquad \texttt{Kd}, \texttt{Bd} \;\Rightarrow\; \hat{\mu}_-. \tag{57}$$

The apparent inversion for bra-side interactions results from the fact that right multiplication of a density matrix is implemented by applying the adjoint operator to the stored bra vector. This implementation preserves the coherence-order convention defined in the next subsection while allowing ket and bra branches to be propagated using ordinary Hilbert-space vectors.

2.6.2 Coherence-order history

For a dyad $|N_{\mathrm{ket}}, a\rangle \langle N_{\mathrm{bra}}, b|$, the signed coherence order is

$$q = N_{\mathrm{ket}} - N_{\mathrm{bra}}, \tag{58}$$

where the two N values are excitation-manifold indices, not counts of ket- and bra-side interactions. The coherence-order change associated with each interaction is

$$\Delta q(\texttt{Ku}) = +1, \qquad \Delta q(\texttt{Kd}) = -1, \qquad \Delta q(\texttt{Bu}) = -1, \qquad \Delta q(\texttt{Bd}) = +1. \tag{59}$$

For an ordered interaction sequence $(I_1, I_2, \ldots, I_N)$, the coherence order after interaction n is

$$q_n = \sum_{m=1}^{n} \Delta q(I_m), \qquad q_0 = 0. \tag{60}$$

The complete coherence-order history is therefore

$$(q_1, q_2, \ldots, q_N). \tag{61}$$

This history is automatically generated from the interaction labels when `coherence_orders` is omitted. If a coherence-order tuple is provided explicitly, it must exactly match the values implied by the interaction sequence.

The coherence-order history is used by `generate_nq_spectrum` to separate $+NQ$, $-NQ$, and total NQ contributions.

2.6.3 FrequencyPathway fields

A complete pathway is defined by

```
FrequencyPathway(
    name,
    interactions,
    component="custom",
    amplitude=1.0,
    prefactor=None,
    coherence_orders=(),
    detection="polarization",
    origin="user",
)
```

The number of interactions in a pathway must match the number of intervals in the associated `SpectroscopyProtocol`. Each interaction is paired with one propagation interval.

The component name is normalized to lower case. The nonrephasing aliases are internally mapped to the canonical unrephasing component. Other names may be used for custom groupings. The components `"rephasing"` and `"unrephasing"` are validated at third order: a pathway

whose signature (Sec. 2.6.5) is not $(-,+,+)$, respectively $(+,-,+)$, raises `ValueError`, because summing pathways of different signatures into one component mixes phase-matching directions. The field `origin` is `"user"` for a pathway written by hand, `"enumerated"` for those returned by `pathways_from_signature` and `"ufss"` for those of `translate_ufss_diagrams`; it only decides which vanishing pathways are reported (Sec. 2.6.6). The property `signature` and the metadata expose the signature.

2.6.4 Pathway response prefactor

If an explicit `prefactor` is provided, it is used directly as the complex multiplier of the pathway response. Otherwise, the solver constructs the response prefactor from the pathway amplitude, the number of interactions, and the number of bra-side interactions.

Let N_{int} be the total number of interactions and N_{bra} the number of bra-side interactions. The default prefactor is

$$c_p = a_p \, i^{N_{\mathrm{int}}} (-1)^{N_{\mathrm{bra}}}, \tag{62}$$

where a_p is the complex value of `amplitude`.

More generally,

$$c_p = \begin{cases} \texttt{prefactor}, & \text{if an explicit prefactor is supplied,} \\ \texttt{amplitude}\, i^{N_{\mathrm{int}}} (-1)^{N_{\mathrm{bra}}}, & \text{otherwise.} \end{cases} \tag{63}$$

The final pathway response and all associated observables are multiplied by c_p. Therefore, experimental field phases or alternative diagrammatic sign conventions should be encoded through `amplitude` or an explicit `prefactor`.

2.6.5 Signatures and completeness

The pulse j of a pathway radiates with the wave-vector sign $s_j = \Delta q_j$ ($+1$ for `Ku` and `Bd`, -1 for `Kd` and `Bu`), so that the signal is emitted in the direction $\sum_j s_j \boldsymbol{k}_j$. The tuple $(s_1, \ldots, s_n)$ is the signature of the pathway, returned by `pathway_signature(interactions)` and by the property `FrequencyPathway.signature`; `Interaction.sign` gives s_j. Rephasing is $(-,+,+)$, non-rephasing $(+,-,+)$ and the fifth-order double-quantum signal of Example 3 $(-,-,+,+,+)$.

The signal of a signature is the sum over *all* pathways with that signature, and a set written by hand can miss some. A sequence that vanishes for a ground-state model without relaxation can be non-zero with relaxation or with a thermal reference state: for the open two-level system, (B_u, K_u, K_u) is non-zero because the radiative channel moves population to $|g\rangle$ during t_2, and it refills the ground-state bleach. The function

```
pathways_from_signature(
    signature,
    *,
    component="custom",
    detection="polarization",
    operator="light_matter",
    max_manifold=None,
    min_manifold=None,
)
```

returns the 2^n ordered label sequences of a signature, given as a sequence of ± 1 or as a string such as `"-++"`: each pulse with $s_j = +1$ acts as `Ku` or `Bd` and each pulse with $s_j = -1$ as `Kd` or `Bu`. The pathways are named after their labels (`"Bu-Ku-Bd"`) and carry `origin="enumerated"`. The sequences that the model does not allow vanish when the spectrum is computed, so the set is complete by construction. For the two-level system with relaxation, three of the eight rephasing sequences are non-zero, the pathways R_1, R_2 and R_3 of the manuscript; with a thermal state the other sequences, which start with a ket de-excitation, contribute as well.

The arguments `max_manifold` and `min_manifold` apply the rule 4(a) of the UFSS diagram generator: the ket and bra manifold indices start at 0 (`Ku`, `Bd` change them by $+1$ and -1 on the ket and bra respectively, `Kd` and `Bu` by the opposite signs) and a sequence is dropped when one leaves $[$`min_manifold`, `max_manifold`$]$. With a ground-state reference and `max_manifold=1` the rephasing set reduces to R_1 and R_2. The filter is valid only for a ground-state reference state without relaxation between manifolds; with relaxation or thermal populations leave both arguments at `None`.

2.6.6 Vanishing pathways

A pathway is null when its response vanishes on the whole grid (`max`$|S| \leq$ `tolerance` $\times$ $\max_p \max |S_p|$ with the default tolerance 10^{-14}, the second maximum running over the pathways of the result). `SpectrumResult.null_pathways(tolerance=1e-14)` returns their names and `result.diagnostics["null_pathways"]` stores them. A pathway *written by hand* that is null usually reveals a sequence that the transition blocks of the model do not allow or a final coherence order that the detection operator cannot read (a polarization needs $q = \pm 1$), so `generate_spectrum` emits a `NullPathwayWarning` for it; `warn_null=False` silences the warning. Generated lists (`"enumerated"`, `"ufss"`) are not reported, since their zero members are expected, unless every pathway of the result is null.

2.6.7 UFSS diagram translation

The helper

```
translate_ufss_diagrams(
    diagrams,
    component="custom",
    names=None,
    operator="light_matter",
    amplitudes=None,
    prefactors=None,
    detection="polarization",
)
```

translates UFSS-style diagrams into QuDPy-FDGF pathways. Each diagram is represented as an ordered sequence of pairs

$$(\texttt{label}, \texttt{pulse_index}), \tag{64}$$

for example

```
[
    ("Ku", 1),
    ("Bd", 2),
```

```
    ("Ku", 3),
]
```

The helper converts every pair into an `Interaction`, preserves the interaction order, assigns unique pathway names, and constructs the corresponding `FrequencyPathway` objects.

This translation preserves the UFSS coherence-order convention

$$\Delta q = \{+1, -1, -1, +1\} \tag{65}$$

for the labels $\{\texttt{Ku}, \texttt{Kd}, \texttt{Bu}, \texttt{Bd}\}$, respectively. The translated pathways carry `origin="ufss"`. The UFSS generator removes the diagrams that leave the manifolds allowed by its `maximum_manifold` (its rule 4(a)); when population transfer couples the manifolds, such diagrams can be non-zero, so `maximum_manifold` must then be raised (the rephasing set of a two-level system with radiative relaxation needs `maximum_manifold = 2`), or `pathways_from_signature` used instead.

2.7 `SpectroscopyPlotter`

`SpectroscopyPlotter` is a rendering-only utility for objects returned by `SpectroscopySolver`. It selects, phase-rotates, normalizes, and renders already calculated data; it never changes the numerical result. Matplotlib is therefore an optional dependency of the plotting layer only.

2.7.1 Constructor

The constructor is

```
SpectroscopyPlotter(
    w_list=None,
    detection_phase=0.0,
)
```

The optional `w_list` provides a default frequency axis for plotting raw spectra with `plot_contourf_multi_spectra` or `plot_1d`.

The parameter `detection_phase` specifies a display-phase correction. For a complex signal S, the plotter applies

$$S_\phi = \exp(i\phi_{\mathrm{det}})S \tag{66}$$

before separating real, imaginary, and absolute views. A plotting configuration may override the constructor value locally.

2.7.2 Selecting data from a `SpectrumResult`

The method

```
select_spectrum_result_data(
    result,
    source="pathways",
    names=None,
    pathways=None,
    totals="auto",
    observable=None,
)
```

selects panels from a two-dimensional result. The source may contain pathways, components, observables, or observable components; a user mapping is also accepted. For observable sources, `observable` selects the detector. `names` or the legacy `pathways` argument restricts the panels, and `totals` controls component-total panels. The first result axis is plotted vertically and the second horizontally.

2.7.3 Dictionary-based plotting configuration

The principal entry point is

```
plot_spectrum_result(result, params=None, **overrides)
```

where keyword overrides take precedence over `params`. Important options include the data `source`, selected names, `observable`, view, normalization, detection phase, diagonal guides, quadrant, colormap, contour levels, labels, title, saving, and display. The method returns a `PlotResult` rather than only a Matplotlib figure.

2.7.4 Views, normalization, and guides

The `view` option accepts `"real"`, `"imag"`, `"abs"`, or `"all"`. Normalization may be independent per panel, shared per row, global, or disabled through `"panel"`, `"row"`, `"global"`, and `"none"`; legacy aliases are normalized internally. Real and imaginary panels use symmetric color limits around zero, whereas absolute-value panels use non-negative limits.

Diagonal reference lines may be selected automatically or requested as rephasing, unrephasing, or both. `plot_quadrant` accepts the full plane, a zoomed region, or quadrants 1–4. Convenience wrappers cover pathway panels, real/imaginary/absolute views, result contours, and multiple coherence orders. Raw-array utilities remain available for contour and one-dimensional plots.